

Hybrid Access Control System for Cloud-Based Productivity Services

CS 428/528 — Computer Security | Group Project 4
Domain: Google Workspace (Docs · Gmail · Drive · Calendar)

Apr 17, 2026

1. System Architecture

The system implements a standard **PAP** → **PDP** → **PEP** policy-based access control architecture on top of a Python/Flask web application backed by SQLite. RBAC1 (role hierarchy) and ABAC attribute evaluation are combined inside the PDP to produce a single permit/deny decision for every access request.

PAP Policy Administration Point	PDP Policy Decision Point	PEP Policy Enforcement Point
Admin defines roles, permissions, users, resources & ABAC rules via REST API / UI	Combines RBAC1 hierarchy + 11 ABAC policies → permit/deny decision + reason	Flask /access endpoint intercepts requests, calls PDP, returns decision to client
Flask PAP endpoints /admin/*	pdp.py → rbac.py + abac.py	POST /access Flask + index.html

Figure 1 — Three-tier policy architecture (PAP · PDP · PEP)

1.1 Request Evaluation Flow

1	Client sends POST /access with user_id, resource_id, action, context
2	PEP (Flask) validates input, forwards to PDP evaluate()
3	PDP resolves user attributes (dept, clearance, is_auditor) from SQLite
4	PDP calls check_rbac() — walks RBAC1 hierarchy for inherited permissions
5	PDP calls evaluate_abac_policies() — evaluates all 11 policies in order

6	Decision logic: ABAC deny > RBAC deny > permit; reason string assembled
7	Access event logged to access_logs table (user, resource, decision, reason)
8	JSON response { decision, reason, user, resource, rbac, abac } returned

Figure 2 — Step-by-step access decision sequence

The SQLite database stores six tables: *users*, *roles*, *user_roles*, *resources*, *resource_assignments*, *role_permissions*, *role_exclusions*, and *access_logs*. All RBAC data (hierarchy, exclusions, cardinality limits) is kept in the DB; ABAC policy logic lives purely in *abac.py*.

2. Domain — Google Workspace

The selected domain models a mid-size organisation using Google Workspace. Employees, managers, administrators, contractors, and auditors interact with **Docs**, **Gmail**, **Drive**, and **Calendar**. The system enforces least-privilege access, separation of duties, and context-aware restrictions (time-of-day, device type, login location).

Resources are classified by sensitivity level:

- **public** — visible to all authenticated users
- **internal** — restricted to organisational members by role
- **confidential** — requires high clearance AND matching department

User attributes tracked: department, clearance level (low/medium/high), employment type, *is_auditor*.

3. RBAC Design (RBAC1 — Role Hierarchy)

The system implements **RBAC1**: roles form an inheritance chain where senior roles implicitly hold all permissions of junior roles. The chain is stored via a *parent_role* column on the roles table and resolved recursively at evaluation time in *rbac.py:get_inherited_roles()*.

Role	Inherits From	Max Users	Key Additions
administrator	manager → employee	5	gmail/delete, calendar/delete, system/admin
manager	employee	—	docs/delete, docs/share, drive/delete, drive/share
employee	— (base role)	—	read+write+comment+send on all resources
contractor	— (isolated)	20	read+comment on assigned resources only
viewer	— (isolated)	—	read+comment on docs; read on others

Table 1 — Role hierarchy and key own permissions

3.1 Full Permission Matrix (own + inherited)

Role / Resource	read	write	comment	delete	share	send	admin
admin / docs	✓	✓	✓	✓	✓	.	.
admin / gmail	✓	✓	.	✓	.	✓	.
admin / drive	✓	✓	✓	✓	✓	.	.
admin / calendar	✓	✓	.	✓	.	✓	.
admin / system	.	.	.	.	.	.	✓
manager / docs	✓	✓	✓	✓	✓	.	.
manager / gmail	✓	✓	.	.	.	✓	.
manager / drive	✓	✓	✓	✓	✓	.	.
manager / calendar	✓	✓	.	.	.	✓	.
manager / system	.	.	.	.	.	.	.
employee / docs	✓	✓	✓	.	.	.	.
employee / gmail	✓	✓	.	.	.	✓	.
employee / drive	✓	✓	✓	.	.	.	.
employee / calendar	✓	✓	.	.	.	✓	.
employee / system	.	.	.	.	.	.	.
contractor / docs	✓	.	✓	.	.	.	.
contractor / gmail	.	.	.	.	.	.	.
contractor / drive	✓	.	.	.	.	.	.
contractor / calendar	✓	.	.	.	.	.	.

contractor / system	.	.	.	.	.	.	.
viewer / docs	✓	.	✓	.	.	.	.
viewer / gmail	✓	.	.	.	.	.	.
viewer / drive	✓	.	.	.	.	.	.
viewer / calendar	✓	.	.	.	.	.	.
viewer / system	.	.	.	.	.	.	.

Table 2 — ✓ = permitted · = not permitted (inherited cells included in senior roles)

4. ABAC Policies

Eleven attribute-based policies are evaluated in priority order inside `abac.py:evaluate_abac_policies()`. An explicit ABAC **deny** overrides any RBAC permit. An explicit ABAC **permit** (P1) resolves a narrowing conflict but does not bypass a missing RBAC grant. Context attributes (hour, device_type, login_location) are evaluated in policies P8–P11.

#	Policy	Attribute Type	Effect
P 1	Employee may write Docs only if owner or explicitly assigned	User role + Resource owner/assignment	Permit / Deny
P 2	Contractors: read and comment only — no write/delete/share/send/admin	User role (contractor)	Deny
P 3	Contractors may only access resources assigned to them	User role + Resource assignment	Deny
P 4	Only managers+ may share Drive files	User role	Deny
P 5	Confidential resources require high clearance + matching department	User clearance & dept + Resource sensitivity/dept	Deny
P 6	Gmail send only from @org.com accounts	User email domain	Deny
P 7	SoD: auditors cannot share or admin	User is_auditor flag	Deny

P 8	Contractors: business hours only (09:00–17:00)	Context: hour	Deny
P 9	Personal devices cannot access confidential resources	Context: device_type	Deny
P 10	Remote login cannot perform admin actions	Context: login_location	Deny
P 11	Remote + confidential requires high clearance	Context: login_location + User clearance	Deny

Table 3 — ABAC policies evaluated by the PDP

5. RBAC2 Constraints

RBAC2 adds **static separation-of-duties** constraints and **cardinality limits**. Exclusion pairs are stored in the `role_exclusions` table and checked at role-assignment time in `app.py:assign_role()` via `rbac.py:check_role_exclusions()` and `check_role_cardinality()`. A user cannot be assigned a role that conflicts with one they already hold, and capped roles reject new assignments once full.

Type	Constraint	Detail
SoD Exclusion	administrator ⊗ contractor	Privilege escalation risk
SoD Exclusion	manager ⊗ contractor	Conflict of interest
SoD Exclusion	employee ⊗ contractor	Mutually exclusive roles
SoD Exclusion	viewer ⊗ contractor	Assign only one
SoD (ABAC P7)	auditor cannot share/admin	Role attribute + action guard
Cardinality	administrator: max 5 users	Limit blast radius of admins
Cardinality	contractor: max 20 users	Cap third-party exposure

Table 4 — RBAC2 static SoD exclusions and cardinality limits

6. Demo Scenarios

The following scenarios are available as one-click Quick Scenarios in the simulator UI and exercise every policy path:

Scenario	User	Action/Resource	Expected	Policy
Employee: own doc write	carol	write / Q1 Report	PERMIT	RBAC1+P1
Employee: unowned doc write	carol	write / Confidential Doc	DENY	P1
Employee: comment on doc	carol	comment / Q1 Report	PERMIT	RBAC1
Manager inherits employee read	bob	read / Q1 Report	PERMIT	RBAC1
Manager: share Drive	bob	share / Engineering Drive	PERMIT	RBAC1+P4
Contractor: assigned read	dave	read / Q1 Report	PERMIT	P3
Contractor: unassigned read	dave	read / Public Announcement	DENY	P3
Contractor: write blocked	dave	write / Q1 Report	DENY	P2
Employee: share Drive blocked	carol	share / Engineering Drive	DENY	P4
Confidential: high clearance	bob	read / Confidential Doc	PERMIT	P5
Confidential: low clearance	carol	read / Confidential Doc	DENY	P5
Ext. email: send blocked	dave	send / Org Inbox	DENY	P6
Auditor: share blocked (SoD)	frank	share / Engineering Drive	DENY	P7
Contractor: off-hours	dave	read / Q1 Report (hr=22)	DENY	P8
Personal device + confidential	bob	read / Confidential (dev=personal)	DENY	P9
Remote: admin blocked	alice	admin / system (loc=remote)	DENY	P10
Remote: conf low clearance	carol	read / Confidential (remote)	DENY	P11
RBAC2: SoD exclusion test	(admin)	assign contractor to manager	REJECT	RBAC2

Table 5 — Demo scenarios covering all 11 ABAC policies + RBAC1/RBAC2

7. Conclusion

The system delivers a complete, deployable hybrid RBAC+ABAC access control solution for a cloud productivity domain. Key achievements:

- **RBAC1 hierarchy** — administrator ← manager ← employee inheritance resolved at query time; only delta permissions stored per role.
- **RBAC2 constraints** — four static SoD exclusion pairs plus cardinality caps (5 admins, 20 contractors) enforced at role-assignment time.
- **11 ABAC policies** — covering role narrowing, ownership, clearance, department matching, org email, SoD, time-of-day, device type, and login location.
- **PAP/PDP/PEP architecture** — clean separation of policy administration, decision logic, and enforcement across Flask endpoints and modules.
- **Audit logging** — every access decision (permit/deny) is logged with user, resource, action, reason, and timestamp; viewable only by admins (auth enforced).

- **Interactive UI** — web dashboard with 18 one-click demo scenarios, live role/hierarchy matrix, RBAC2 constraint viewer, and user registration with role assignment.

AI tools (Claude Code) were used for implementation guidance and code generation. All design decisions, policy logic, and architecture were authored and understood by the team.